

Day 2 – Object-Oriented Programming in C++

Classes, Objects, Encapsulation, Inheritance, Polymorphism & Abstraction

1. Classes and Objects

A **class** is a blueprint or template for creating objects. An **object** is an instance of a class.

A **constructor** is a special class member used to initialize values into the class's variables. Key rules for a constructor:

- It must have the same name as the class.
- It does not have a return type.

Program 1: Student Class with Constructor

```
#include <iostream>
using namespace std;

int b; // global variable

class Student {
public:
    string Name;
    int Roll;
    float CGPA;

    Student(string name, int roll, float cgpa) {
        Name = name;
        Roll = roll;
        CGPA = cgpa;
    }

    void display() {
        cout << Name << endl;
        cout << Roll << endl;
        cout << CGPA << endl;
    }
};

int main() {
    Student s("mamo", 117, 8.9);
    cout << s.Name << endl;
    cout << s.Roll << endl;

    Student s1("Raksha", 101, 9.6);
    Student s2("Rakesha", 109, 9.5);
    cout << s2.Name << endl;

    s.display();
}
```

```

        s1.display();
        s2.display();

    return 0;
}

```

2. Encapsulation

Encapsulation is the practice of bundling data and the methods that operate on it within a class, while restricting direct access to some of the object's components (typically by making them **private**).

Program 2: Bank Class – Private Pin with Getter/Setter

```

#include <iostream>
using namespace std;

class Bank {
private:
    int pin = 8868;

public:
    string name = "Karthick";
    int bal = 2000;

    void get_pin() {
        cout << pin << endl;
    }

    void set_pin() {
        pin = 2374;
    }
};

int main() {
    Bank b;
    cout << b.name << endl;
    b.get_pin();
    b.set_pin();
    b.get_pin();

    return 0;
}

```

3. Class Method Using Vectors - Two Sum

A class method can accept and return STL containers such as `vector<int>`, letting a class encapsulate algorithmic logic like the classic Two Sum problem.

```

#include <vector>
using namespace std;

class Solution {
public:

```

```

vector<int> twoSum(vector<int>& nums, int target) {
    int n = nums.size();

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] == target) {
                return {i, j};
            }
        }
    }
    return {};
}
};

```

4. Inheritance

Inheritance is an OOP concept where one class acquires the properties and functions of another class. Common types of inheritance:

- Single-level Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Single-level inheritance: one class inherits from a single parent class.

Program 3: Single-Level Inheritance (Cat → Dog)

```

#include <iostream>
using namespace std;

class Cat {
public:
    void eat() {
        cout << "Cat is eating" << endl;
    }
};

class Dog : public Cat {
public:
    void sound() {
        cout << "Boww boww" << endl;
    }
};

int main() {
    Dog d;
    d.sound();
    d.eat();

    return 0;
}

```

```
}
```

Multiple Inheritance

One child class inherits from more than one parent class.

Program 4: Multiple Inheritance (GrandFather + Father → Daughter)

```
#include <iostream>
using namespace std;

class GrandFather {
public:
    void house() {
        cout << "Grandfather house";
    }
};

class Father {
public:
    void car() {
        cout << "Father car";
    }
};

class Daughter : public Father, public GrandFather {
public:
    void mobile() {
        cout << "My mobile";
    }
};

int main() {
    Daughter d;
    d.car();
    d.house();

    return 0;
}
```

Multilevel Inheritance

A class inherits from another class, which itself inherits from another class, forming a chain of inheritance.

Program 5: Multilevel Inheritance (Camera → Mobile → Smartphone)

```
#include <iostream>
using namespace std;

class Camera {
public:
    void photo() {
        cout << "Take a photo";
    }
}
```

```

};

class Mobile : public Camera {
public:
    void call() {
        cout << "Virat Kohli";
    }
};

class Smartphone : public Mobile {
public:
    void gemini() {
        cout << "Use gemini";
    }
};

int main() {
    Smartphone s;
    s.photo();

    return 0;
}

```

Hierarchical Inheritance

Multiple child classes inherit from the same single parent class.

Hybrid Inheritance

A combination of two or more types of inheritance (for example, hierarchical inheritance combined with multilevel inheritance).

5. Polymorphism

Polymorphism means "one name, many forms." It is divided into two types:

- Compile-time (Compiler) Polymorphism
- Run-time Polymorphism

1) Compile-time Polymorphism - Function Overloading

Same function name, different parameters - resolved at compile time.

Program 6: Calculator - Function Overloading

```

#include <iostream>
using namespace std;

class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }
}

```

```

        int add(int a, int b, int c) {
            return a + b + c;
        }
    };

    int main() {
        Calculator c;
        cout << c.add(10, 20, 40);

        return 0;
    }

```

2) Run-time Polymorphism - Virtual Functions

Same function name, but different classes - resolved at run time using virtual functions and base-class pointers.

Program 7: Run-time Polymorphism (Animal → Dog)

```

#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() {
        cout << "Animal sounds" << endl;
    }
};

class Dog : public Animal {
public:
    void sound() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Animal *a;
    Dog d;
    a = &d;
    a->sound();

    return 0;
}

```

6. Abstraction

Abstraction means hiding implementation details and showing only the essential features to the user.

Program 8: Scooty - Abstraction Example

Note: `engine()` is kept private to show that the implementation detail is hidden from the user – only `start()` is exposed publicly.

```
#include <iostream>
using namespace std;

class Scooty {
public:
    void start() {
        cout << "Turn on the key and press self";
    }

private:
    void engine() {
        cout << "It's hidden part";
    }
};

int main() {
    Scooty s;
    s.start();
    // s.engine(); -> not accessible here since engine() is private,
    //               which is exactly what abstraction demonstrates

    return 0;
}
```